Phase 2 Information Architecture Blueprint

Weber State University Class Registration

Kaden Donaldson Human Computer Interaction November 6, 2026

GitHub repository: https://github.com/Kaden-13/Contextual-Research (see the `IA_Mapping` directory)

1. Summary

Phase 1 showed that Weber State students do not experience class registration as one system. To register for one class, a student moves through the eWeber portal, a semester and tuition agreement page, a term picker, the three pane Register for Classes page, CatTracks in a separate tab for prerequisites, and often an advisor. The deepest path to a section's details is 7 levels.

This phase tested how students actually group registration tasks with an open card sort, then rebuilt the structure around their groups.

Card sort findings. Three Weber State students sorted the 20 cards: two did an open sort and one did a closed sort to check the categories. All three put 6 pairs of cards together every time: search with seats, degree progress with prerequisites, add with submit, weekly schedule with overlap checking, paying with the tuition rules, and advisor with the glossary. The biggest split was over class days and times. The freshman treated them as part of searching, while both seniors treated them as part of planning a schedule.

The revised Information Architecture (IA) puts every task at most 2 levels from Registration Home, under 5 primary navigation items. It brings eligibility and degree progress into the same system, separates pending classes from registered ones, and keeps the tuition and drop rule next to the Register button instead of hiding it before class selection.

Two critical flows from Phase 1 were redesigned and wireframed across 6 screens: **finding a section that fits a schedule** and **checking eligibility, then registering**. Every wireframe is built as semantic HTML and annotated with its exact keyboard tab order and ARIA landmarks, roles, and states.

2. Card Sorting Study

2.1 Card Deck

I pulled 20 cards from the Phase 1 audit. Each one is a task a student does around registration, and each traces back to a Phase 1 finding (`card_deck.csv`, column `phase1_source`).

Cards use plain student language (“Add a class to my list”) instead of system names (“Summary pane,” “CRN lookup”), and they avoid repeating keywords, so participants would group by meaning instead of matching words.

ID	Card	Phase 1 source
C01	Search for a class by subject or course number	Task T1, HE09
C02	Compare sections of the same class	Task T2, HE06
C03	See what days and times a class meets	Theme 1, HE13
C04	Check how many seats are open	HE05
C05	Join a waitlist for a full class	HE02, HE05
C06	Check if I meet the prerequisites	Theme 2, HE14
C07	See my progress toward my degree	Theme 2 (CatTracks)
C08	Add a class to my list	Theme 3, HE01
C09	Submit my registration	Theme 3, HE01, HE04
C10	Drop a class	HE08
C11	See my weekly schedule	Weekly calendar (works well)
C12	See which classes I’m actually registered for	HE01, HE10
C13	Find out if two classes overlap	Time Conflict warning (works well)
C14	Choose a semester	Term selection step
C15	Read the tuition and drop rules	HE03
C16	Check for holds on my account	Registration home

ID	Card	Phase 1 source
C17	Find out when I'm allowed to register	Registration home
C18	Pay tuition or see what I owe	HE03, drop policy
C19	Get in touch with my advisor	Theme 4
C20	Look up what a term means (like CRN)	HE07

2.2 Method

Sort type: open. Participants created and named their own groups. Phase 1 found that students do not think of CatTracks, Register for Classes, and tuition as separate tools, so a closed sort using the current menu names would have assumed the very structure being questioned.

Participants. Three Weber State students with a range of registration experience took part on November 6, 2026.

ID	Year	Times registered at Weber State	Sort type	Format
P1	Freshman	2	Open	Remote, text message
P2	Senior	5 to 6	Open	Remote, text message
P3	Senior	7	Closed	Remote, text message

Procedure. P1 and P2 got the same written instructions and the 20 cards as a numbered list. They sorted the cards into groups that made sense to them, named each group as if it were a menu label, put anything that didn't fit in an "Unsure" group, and said which card was hardest to place.

P3 did a **closed sort**. They got the same cards plus 6 category names drawn from P1 and P2's labels (Find Classes, Register, My Schedule, My Degree, Finances, Help) and a "None of these" option. This is a standard way to check whether categories that came out of an open sort make sense to someone who didn't create them.

Remote sorting by text let participants work on their own schedule without the facilitator in the room. Raw replies and session notes are in `01_card_sort/raw_data`.

Limitations. Three participants is the minimum, so the agreement percentages show a direction, not a statistically reliable result. Combining open and closed sorts in one similarity matrix is common for hybrid studies, but P3 could only choose among the given categories.

Their sort tests the categories rather than inventing new ones. Two of the three participants are seniors, so newer students are underrepresented.

Analysis. Results were entered into `raw_data/card_sort_results.csv`, one row per card per participant. A Python script (`analyze_card_sort.py`) builds a similarity matrix showing the percent of participants who placed each pair of cards in the same group. It then runs average linkage hierarchical clustering to produce a cluster diagram, and cuts the tree at 50% agreement to define consensus clusters.

2.3 Results

Groups created. P1, the freshman, made 7 narrow, descriptive groups (for example “Course Search & Exploration” and “Registration & Account Information”). P2, a senior, made 4 broad, one word groups: “register,” “schedule,” “degree,” and “finances.” P3 used all 6 provided categories and did not use “None of these.” No participant used an Unsure pile.

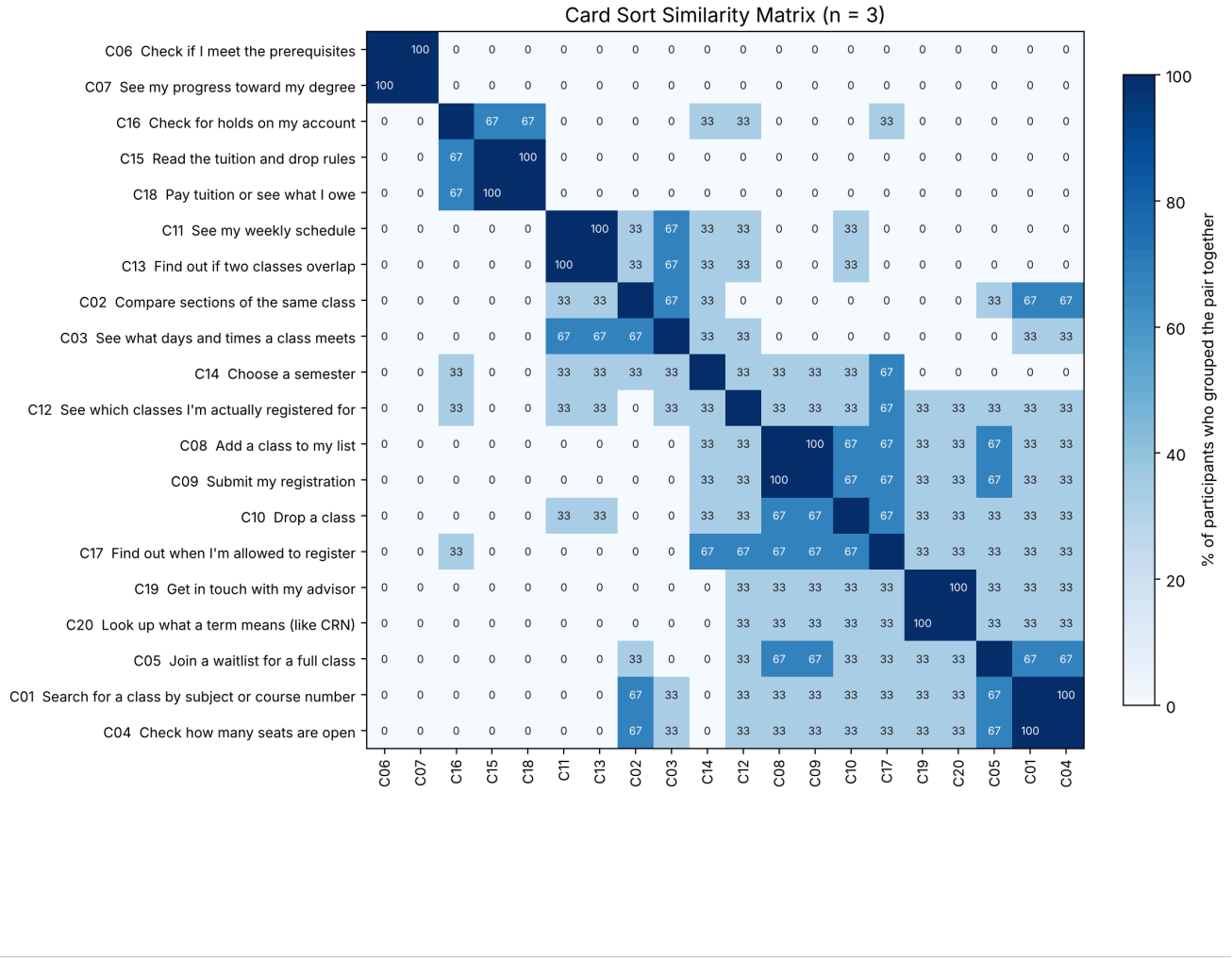


Figure 1. Similarity matrix. Each cell is the percent of participants who put that pair of cards in the same group (0, 33, 67, or 100 with n = 3).

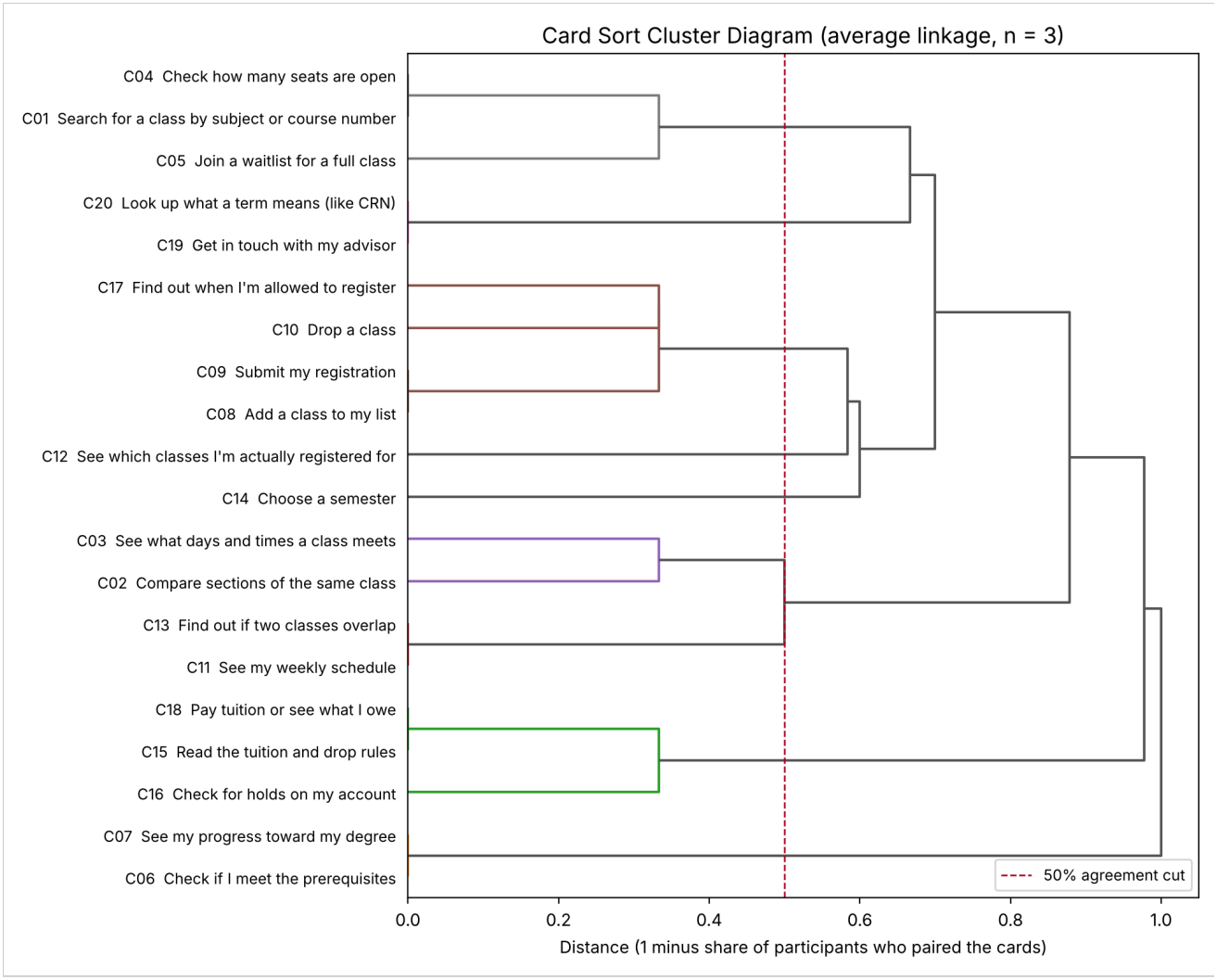


Figure 2. Cluster diagram. The red line marks the 50% agreement cut.

Consensus summary.

Cluster	Cards	Agreement	Participant labels	In the revised IA
Degree	C06 prerequisites, C07 degree progress	100%	Degree Planning & Requirements; degree; My Degree	**My Degree**
Help	C19 advisor, C20 glossary	100%	Advising & Support; register; Help	**Utility nav**, side by side
Finances	C15 tuition and drop rules, C18 pay, C16 holds	78% (C15 + C18 at 100%)	Policies & Finances; finances; Finances	**Finances**

Cluster	Cards	Agreement	Participant labels	In the revised IA
Finding classes	C01 search, C04 seats, C05 waitlist	78% (C01 + C04 at 100%)	Course Search & Exploration; register; Find Classes	**Find Classes**; Join waitlist on each full section
Registering	C08 add, C09 submit, C10 drop, C17 registration date	72% (C08 + C09 at 100%)	Enrollment & Registration; register; Register	Spread across Find Classes, My Schedule, and Registration Home (see the open issue below)
Schedule	C11 weekly schedule, C13 overlaps, C02 compare sections, C03 days and times	61% (C11 + C13 at 100%)	Schedule Management; schedule; My Schedule	**My Schedule**, plus a week view beside the search results
No consensus	C14 choose a semester	3 different groups	account information; schedule; Register	Global semester switcher in the header
No consensus	C12 registered vs pending	67% with C17	account information; register; My Schedule	**My Schedule** (Pending and Registered tables) and a status card on Home

Common categorization patterns

- **Degree and prerequisites belong together, and apart from registration.** All three participants grouped them identically. Today, prerequisite checking lives in CatTracks, which Phase 1 found students open in a separate tab. Students expect it in one place called “degree.”
- **Money is its own space.** All three paired the tuition and drop rules with paying, not with registering. That supports a Finances section. It also helps explain Phase 1 finding HE03: students don’t look for the drop rule while they are registering, so the redesign repeats it next to the Register button.
- **Add and submit are one action.** All three grouped them together, so the redesign keeps the path from Add to Register short, with a “Review and register” link on every Find Classes page.
- **Advisor and glossary are both “help.”** All three paired C19 with C20, which supports putting them side by side in the utility nav.

Label disagreements

- **“Registration” means two different things.** P1 split it into Enrollment & Registration (doing it) and Registration & Account Information (checking status). P2 used one word, “register,” for almost everything. The redesign follows P1’s split: doing happens in Find Classes and My Schedule, and checking status happens on Registration Home.
- **Holds (C16)** went to finances for 2 of 3 participants and account information for 1. Holds appear on Registration Home, because a hold blocks registration, and are also listed under Finances.
- **Choose a semester (C14)** landed in a different group for every participant. With no agreed home, it becomes a global header switcher reachable from every page instead of a step inside one section.
- **Drop a class (C10)** went to register for 2 of 3 and schedule for 1. Drop lives in My Schedule’s Registered table, next to the Register button.

Unexpected mental models

- **Seniors see class times as schedule planning, not search.** Both seniors put “See what days and times a class meets” with schedule, while the freshman put it with search. An experienced student isn’t asking “what sections exist?” but “which section fits my week?” The redesign keeps a live week view in the My Schedule sidebar on the Find Classes page and flags conflicts on each result before Add.
- **Experience shrinks the menu.** The freshman needed 7 categories, but the most experienced open sorter needed only 4. Five primary nav items sits between them. That’s enough to keep degree and finances separate, which everyone wanted, without the fine split only the newer student made.
- **The hardest cards were the “obvious” ones.** P1 found the glossary (C20) hardest, and P3 found degree progress (C07) hardest. Yet every participant put those cards in the same group. Hesitating doesn’t mean a card has no clear home. These are tasks students rarely think about during registration, which supports showing them in context (an inline CRN definition and prerequisite status on each result) instead of only in their own section.

How the data tested the draft IA

I sketched a draft sitemap from the Phase 1 findings before running the sort, then checked each decision against the data.

Draft decision	What the sort showed	Result
Primary item “Tuition and Billing”	The open sorters named this group “finances,” and P3 confirmed it	**Changed:** renamed to Finances

Draft decision	What the sort showed	Result
Holds shown only on Registration Home	2 of 3 put holds with finances	**Changed:** holds stay on Home and are also listed under Finances
My Degree as its own primary item	100% agreement from all three participants	**Confirmed**
Week view sidebar and conflict flags on Find Classes	Both seniors grouped times and section comparison with schedule	**Confirmed and promoted:** treated as a core part of Find Classes, not a convenience
Semester as a global header switcher	No two participants agreed on where it belongs	**Confirmed**
Advisor and glossary in utility nav	100% agreement	**Confirmed**
Join waitlist on each full section in Find Classes	P3 placed the waitlist under Find Classes	**Confirmed**
Registering actions split between Find Classes (Add), My Schedule (Submit, Drop), and Home (registration date)	Add, submit, drop, and registration date form a 72% cluster	**Open issue** for Phase 3 (below)

Open issue for Phase 3. The data shows students think of “register” as one place, but my IA spreads registering across three pages. I kept the split because each action sits where its context is: Add next to the section, and Submit next to the pending list and drop rule. The pending count badge and “Review and register” links tie them together. Phase 3 usability testing should check whether students find the Register button without help, and whether a nav label such as “Schedule and Register” works better than “My Schedule.”

3. Revised Sitemap and IA Blueprint

3.1 Current Structure

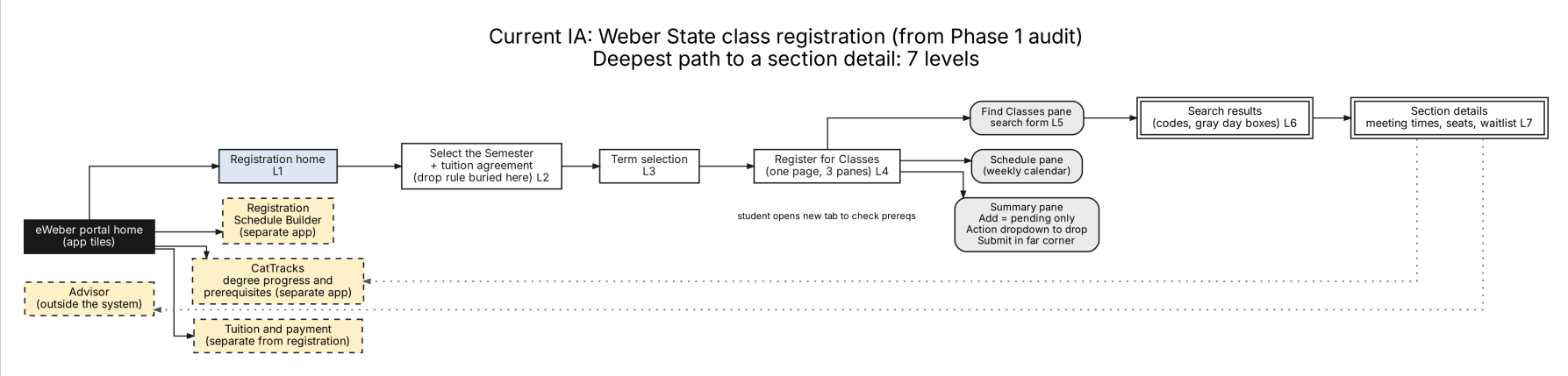


Figure 3. Current IA. Dashed yellow boxes are separate systems the student has to leave registration to use.

3.2 Revised Structure

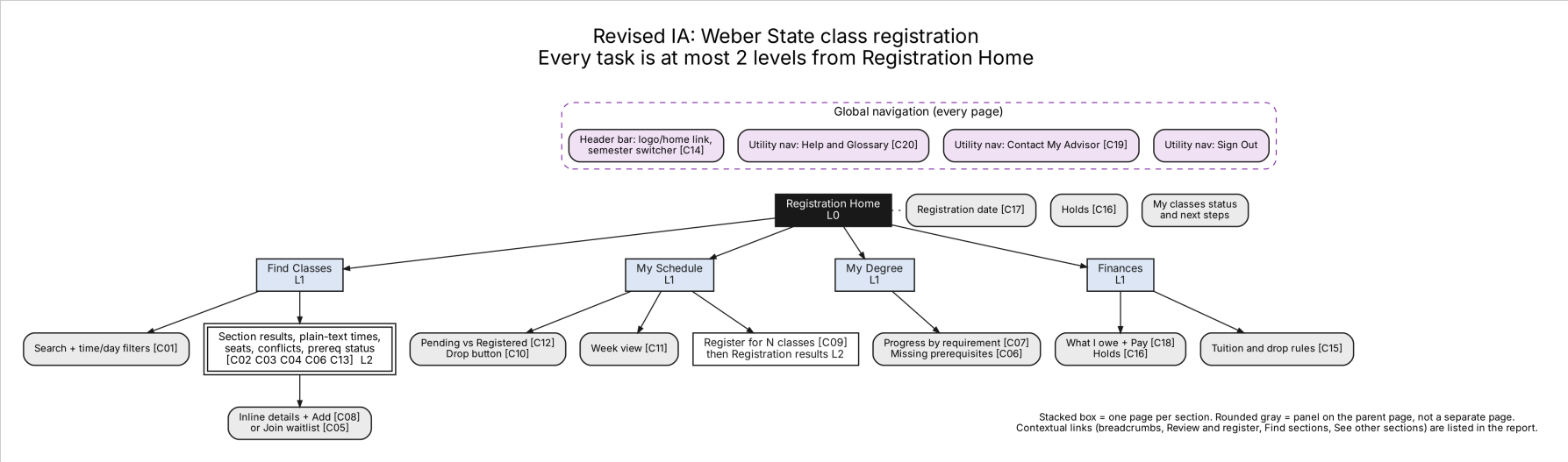


Figure 4. Revised IA. Card IDs in brackets show which card sort items each page serves.

Notation. Solid boxes are pages. The double-bordered box is a set of similar pages (one result per section). Rounded gray boxes are panels on their parent page, not separate pages. Purple boxes are global elements on every page.

3.3 Navigation Hierarchy

Level	Element	Items
Primary (global)	Horizontal nav bar under the header, on every page	Registration Home, Find Classes, My Schedule (with pending count), My Degree, Finances
Secondary (in page)	Section headings and panels inside each primary page; no sub-menus	Find Classes: search, results, details. My Schedule: Pending, Registered, Week view. Finances: what I owe, holds, tuition and drop rules
Utility (global)	Right side of the header bar	Help and Glossary, Contact My Advisor, Sign Out
Global control	Header bar	Semester switcher (replaces the separate term selection page)

3.4 Reducing Depth

Task	Current depth	Revised depth	How
See a section's meeting times	7 (portal, reg home, semester, term, Register for Classes, results, details)	2 (Find Classes, results with times shown)	Semester moved to a header switcher; times shown in each result instead of a details view
Check prerequisites	Leave the system (CatTracks tab or advisor)	1 to 2 (status on each result; My Degree for detail)	Prerequisite status in results; degree progress is a primary nav item
Confirm what I'm registered for	Read the Summary pane's pending vs registered labels	1 (My Schedule)	Pending and Registered are separate, labelled tables
Find the drop rule	Read legal text before choosing classes	0 (next to the Register button) and 1 (Finances)	Repeated at the moment of decision

No primary item has a sub-menu. Anything that used to be a pane, tab, or extra page is now a labelled section inside one primary page, which removes the nested levels that caused the novice participant in Phase 1 to ask "Why do I have to click so many buttons."

3.5 Global and Contextual Navigation

Global navigation (every page):

Element	Purpose
Skip link	First tab stop; jumps to `<main>`
Header bar	Home link, semester switcher, utility nav
Primary nav bar	The 5 top level sections; current page marked with `aria-current="page"` and a thick underline
Pending count badge	Shows how many classes are added but not registered, on every page
Footer	Accessibility statement, registration deadlines, contact the Registrar

Contextual navigation (where it is needed):

Element	Where	Goes to
Breadcrumb	Every page below Home	Back up the path, for example Registration Home / My Schedule / Results
My Schedule sidebar with “Review and register (2)”	Find Classes	My Schedule review
“Review and register” link in the Add confirmation	Find Classes after Add	My Schedule review
“Find sections of CS 2420”	My Degree, missing requirement row	Find Classes filtered to that course
“See other MATH 1210 sections”	Registration results, full section	Find Classes for that course
“Join waitlist”	Full sections and the results page	Adds to waitlist, shown in My Schedule
“What is this?”	Next to CRN in section details	Inline definition
“Contact my advisor”	My Degree help section	Advisor contact

4. User Flows

I chose the two flows that caused the most trouble in Phase 1.

Flow A, finding a section that fits a schedule, was the only task a participant could not finish alone. The novice needed help finding class times, and the experienced student called it the hardest task.

Flow B, checking eligibility and then registering, combines the two costliest problems: prerequisites living in a separate tool, and Add not actually registering the student.

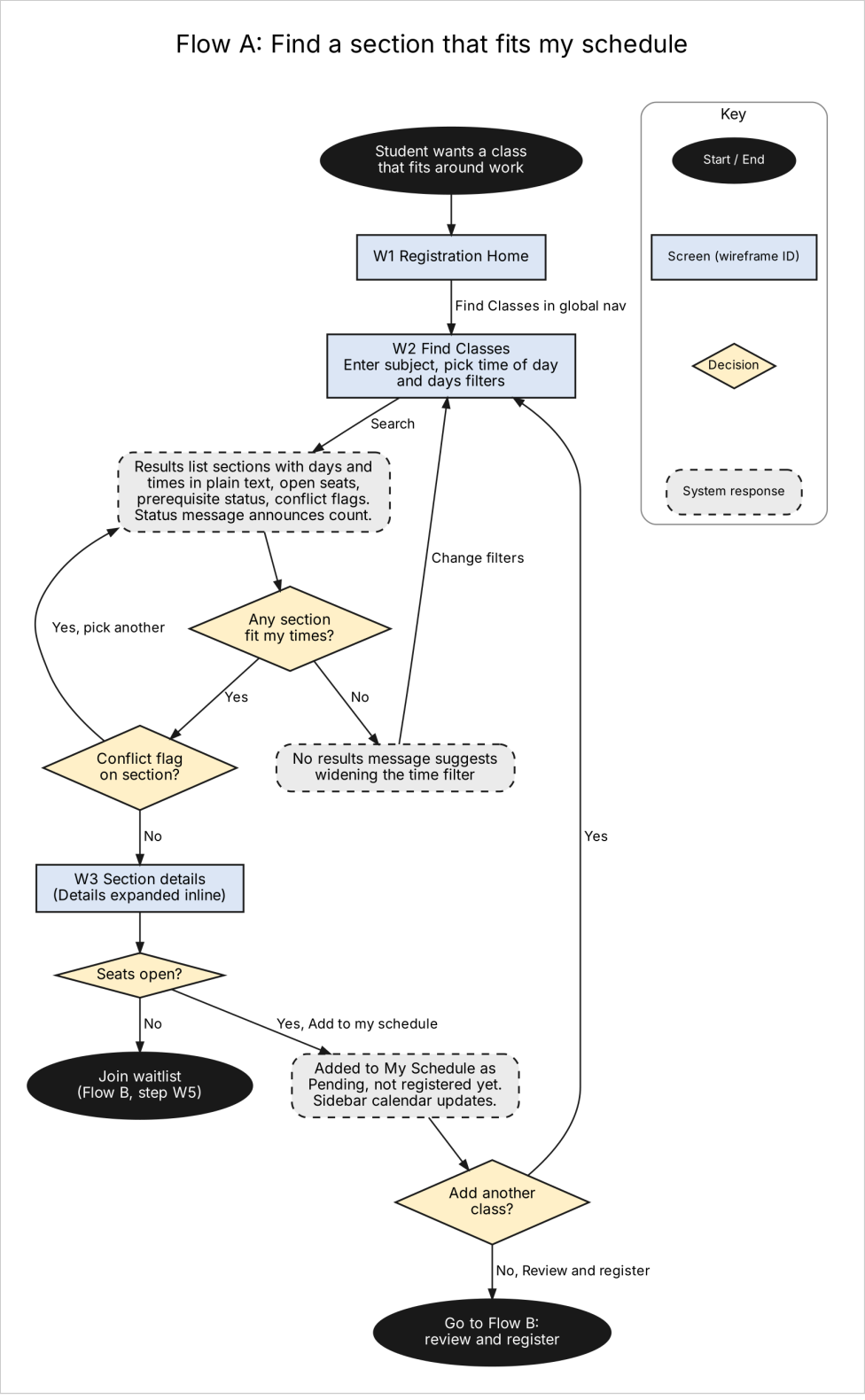


Figure 5. Flow A: Find a section that fits my schedule.

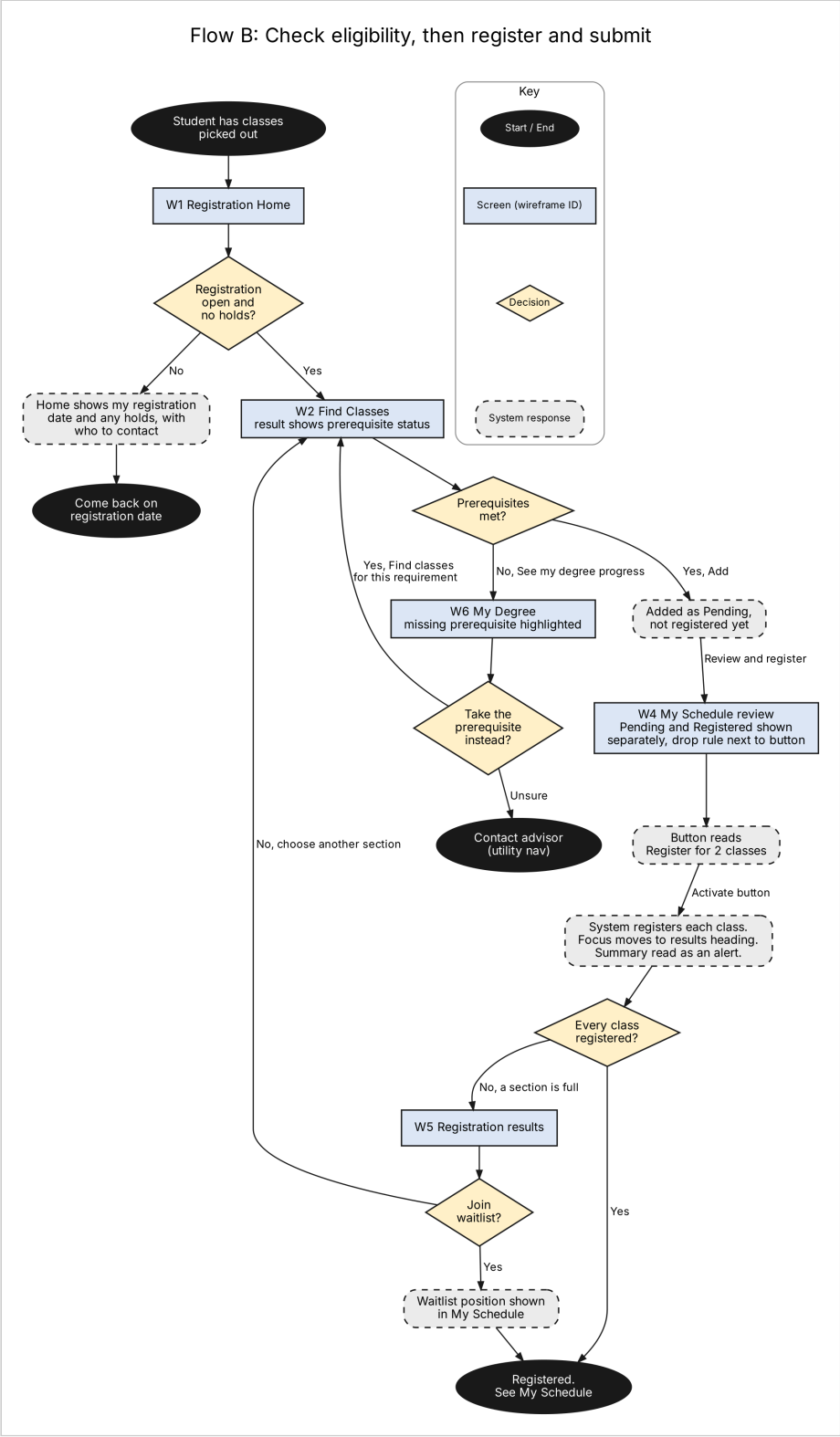


Figure 6. Flow B: Check eligibility, then register and submit.

Phase 1 Problems Each Flow Fixes

Phase 1 finding	Where the flow fixes it
Theme 1 / HE13: can't find class times	Flow A, W2: days and times are the first, bold fact in every result
Theme 2 / HE14: prerequisites live in CatTracks	Flow B, W2 and W6: prerequisite status on each result; My Degree one click away
Theme 3 / HE01: Add looks like registering	Both flows: "Added, not registered yet" status; separate Pending table; button names the count
HE02: full class error is vague	Flow B, W5: names the full section and offers waitlist or other sections
HE03: drop rule buried	Flow B, W4: drop rule right next to the Register button and linked to it with `aria-describedby`
A01: keyboard users can't reach Add	All screens: Add is a real `<button>` in normal tab order

5. Wireframes

All 6 screens are medium fidelity grayscale wireframes built in HTML and CSS (`IA_Mapping/04_wireframes/src`). Building them as real HTML means the semantic structure and tab order in Section 6 come straight from the markup instead of being drawn by hand. Clean and annotated exports of every screen are in `04_wireframes/exports`.

How to read the annotations:

- **Red numbered circles** show keyboard tab order.
- **Colored dashed outlines** with a tag show each landmark and its role.
- **Yellow tags** show ARIA states and live regions.

W1: Registration Home

Answers the three questions students had to hunt for in Phase 1: when can I register, do I have holds, and what is my status. The drop rule appears here in one plain sentence instead of legal text.

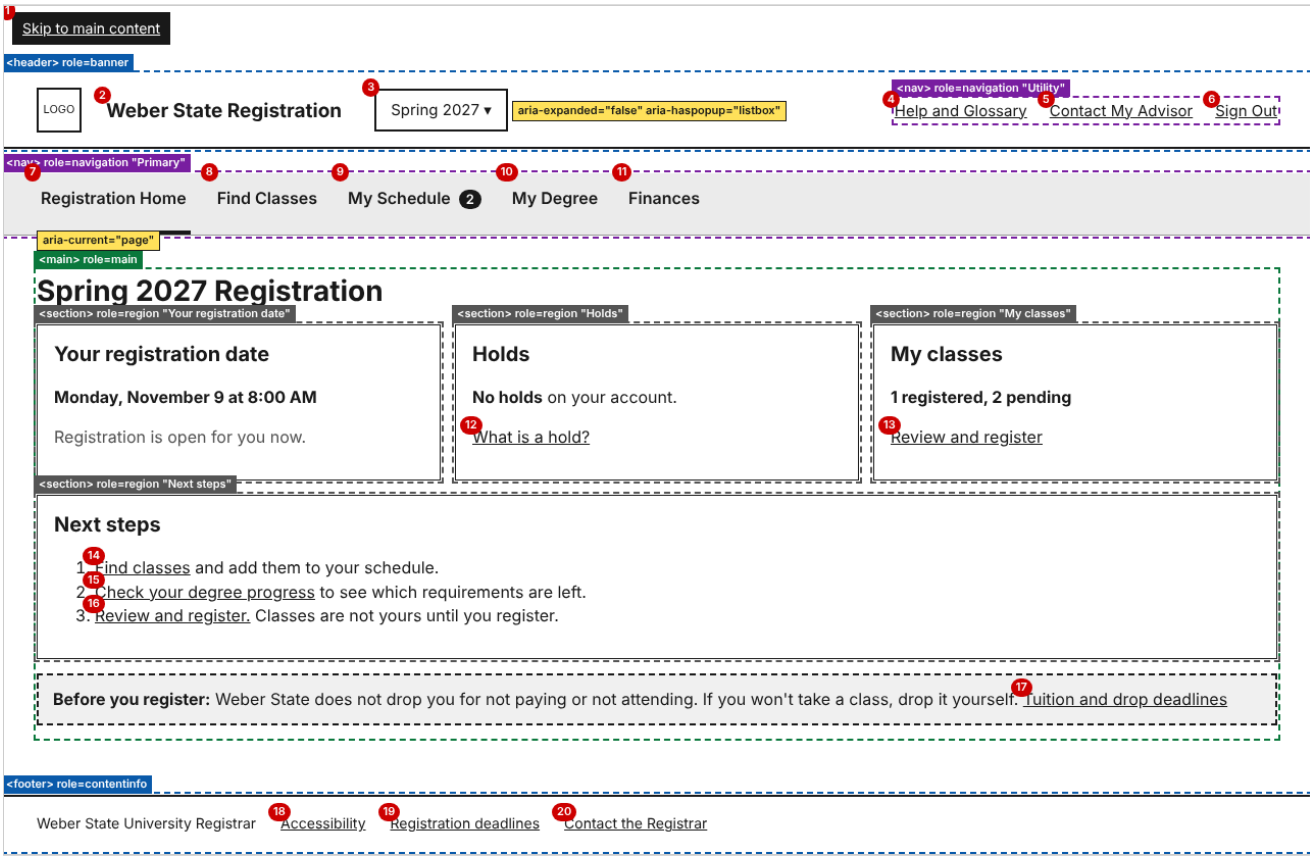


Figure 7. W1 Registration Home, annotated.

W2: Find Classes

Each section shows days and times in bold plain text, location, open seats, and prerequisite status in one row. Time of day filters support Flow A. Conflicts are flagged before Add. The My Schedule sidebar always shows what is pending.

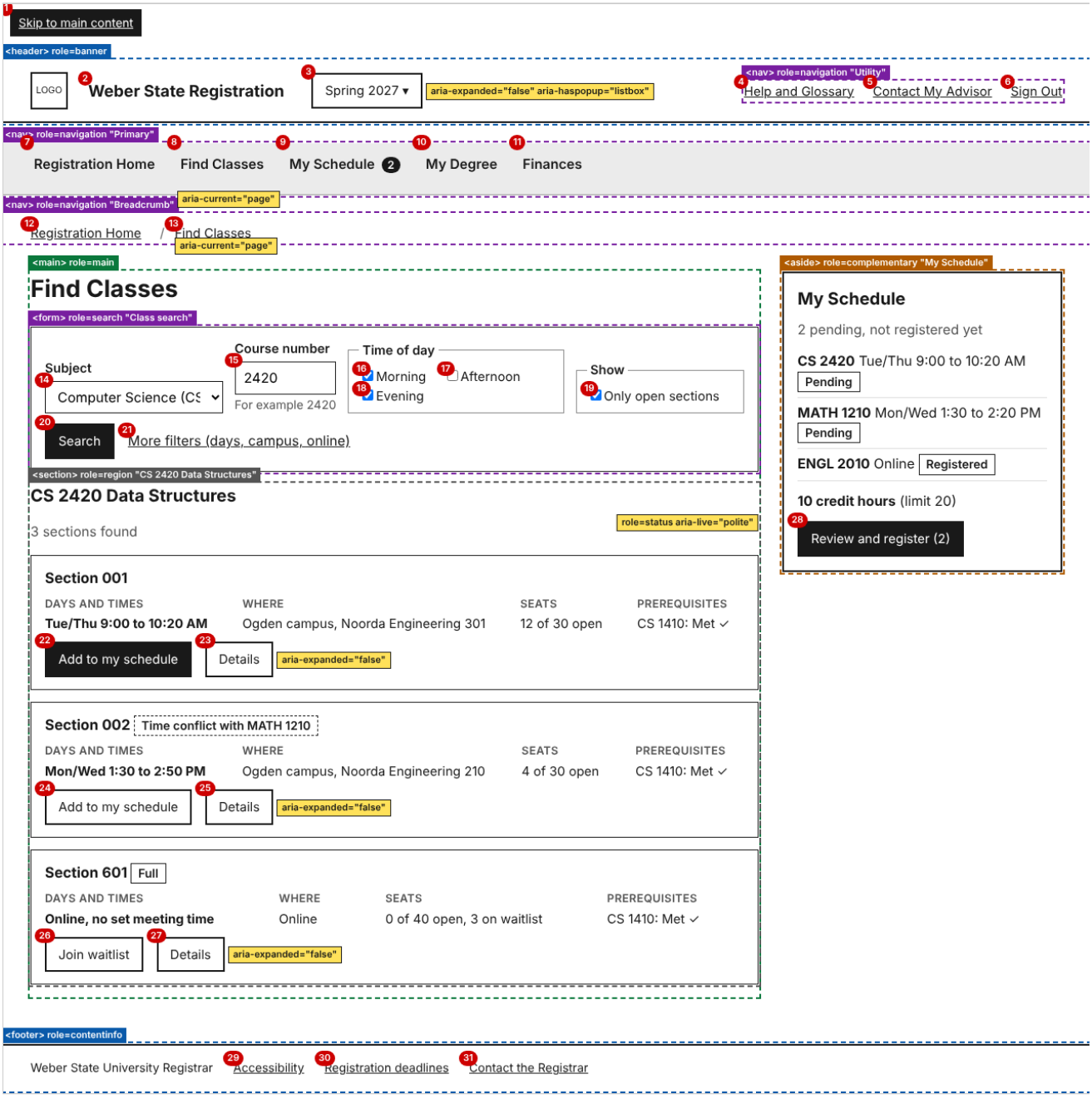


Figure 8. W2 Find Classes, annotated.

W3: Find Classes after Add

After Add, a status message says plainly that the class is not registered yet and links to Review and register. Details expand inline with a plain definition of CRN.

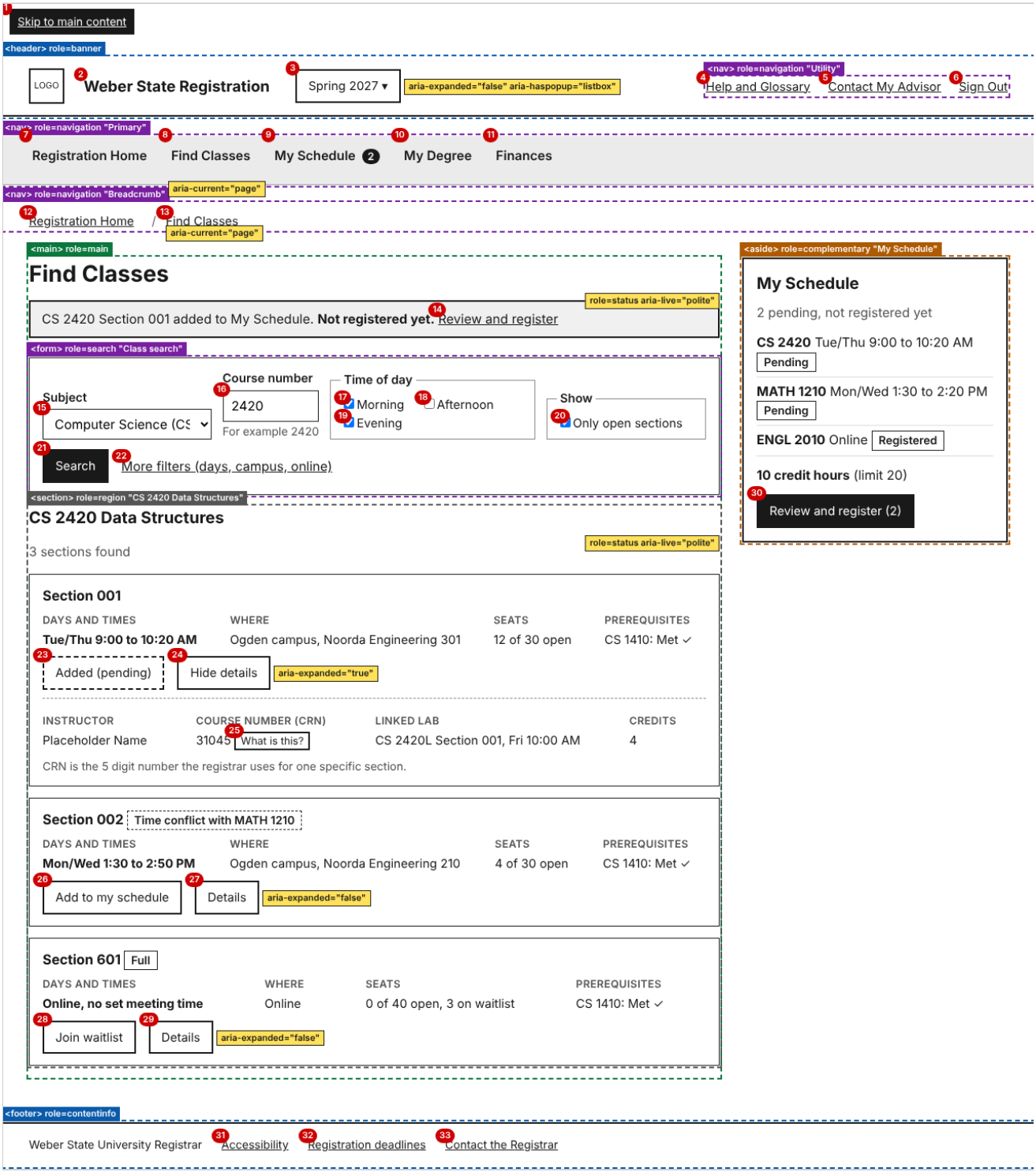


Figure 9. W3 Find Classes after Add, annotated.

W4: My Schedule review

Pending and Registered are separate tables with plain headings. The drop rule sits directly above the button, which says exactly what it will do: Register for 2 classes. Drop is a visible button, not a dropdown.

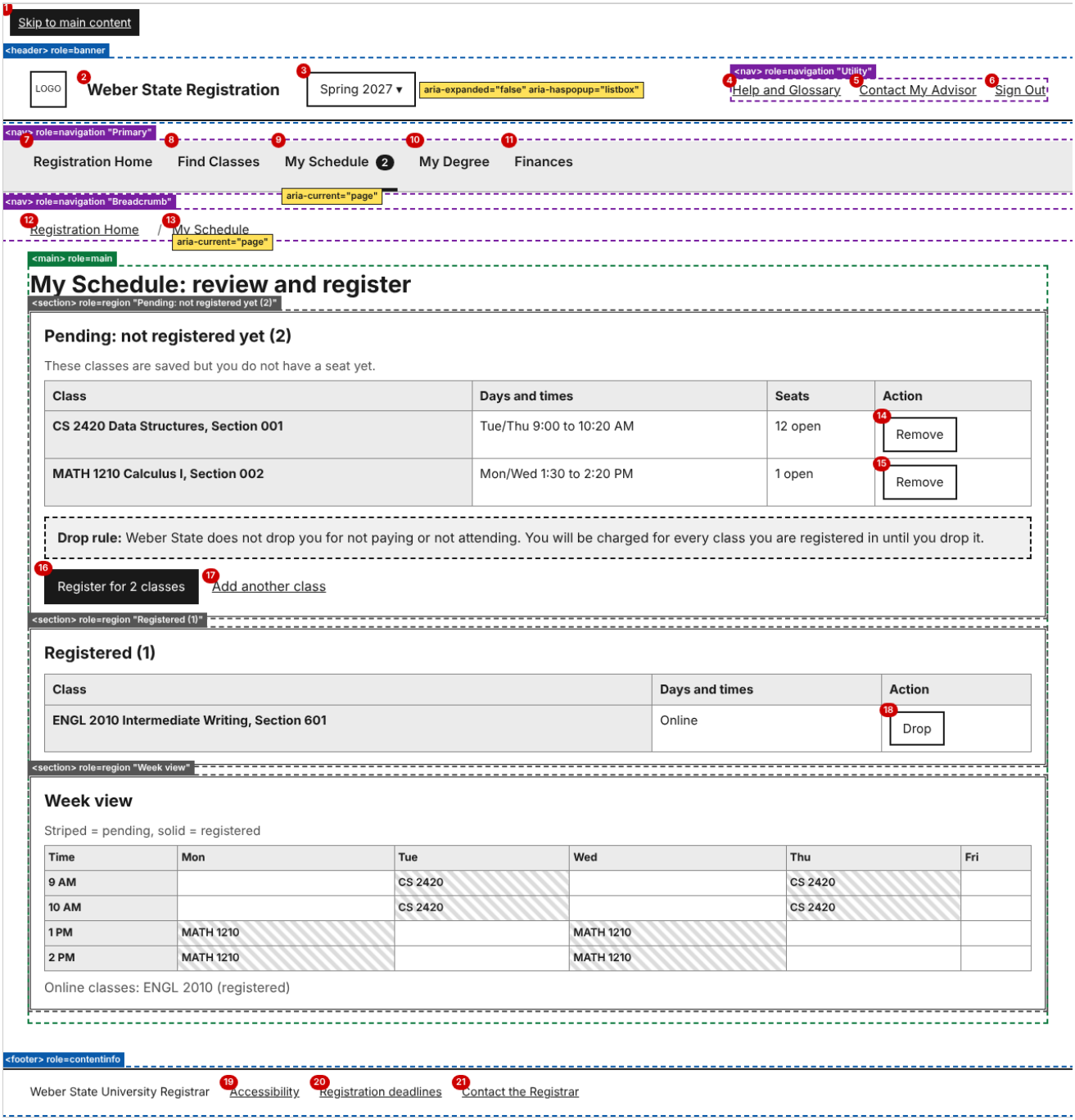


Figure 10. W4 My Schedule review, annotated.

W5: Registration results

Focus moves to this heading after Register. The alert summarizes the outcome, and the full section error names the problem and offers two ways forward.

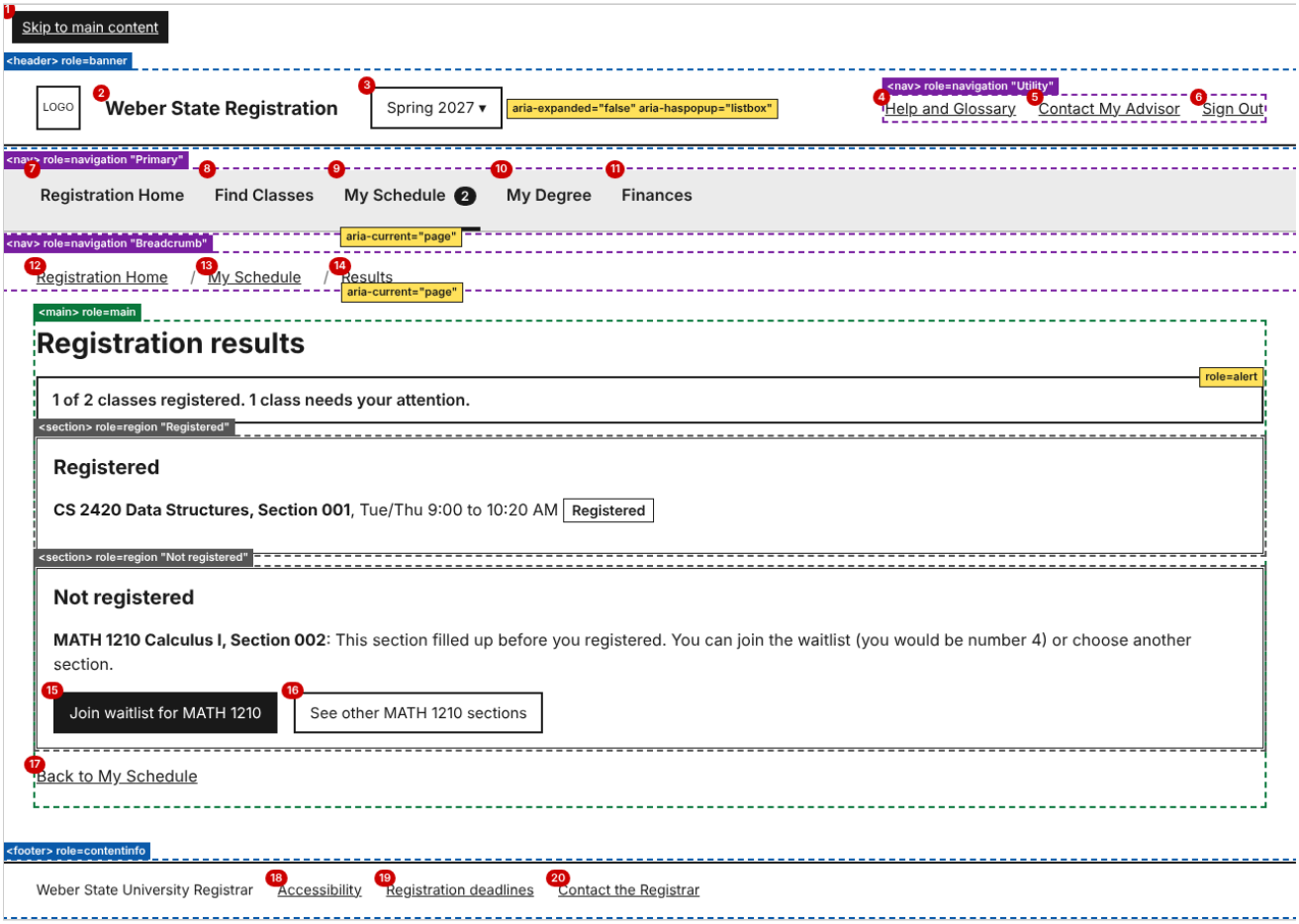


Figure 11. W5 Registration results, annotated.

W6: My Degree

Brings CatTracks style progress into the same navigation. When a prerequisite is missing, the row links straight to sections of the course the student needs.

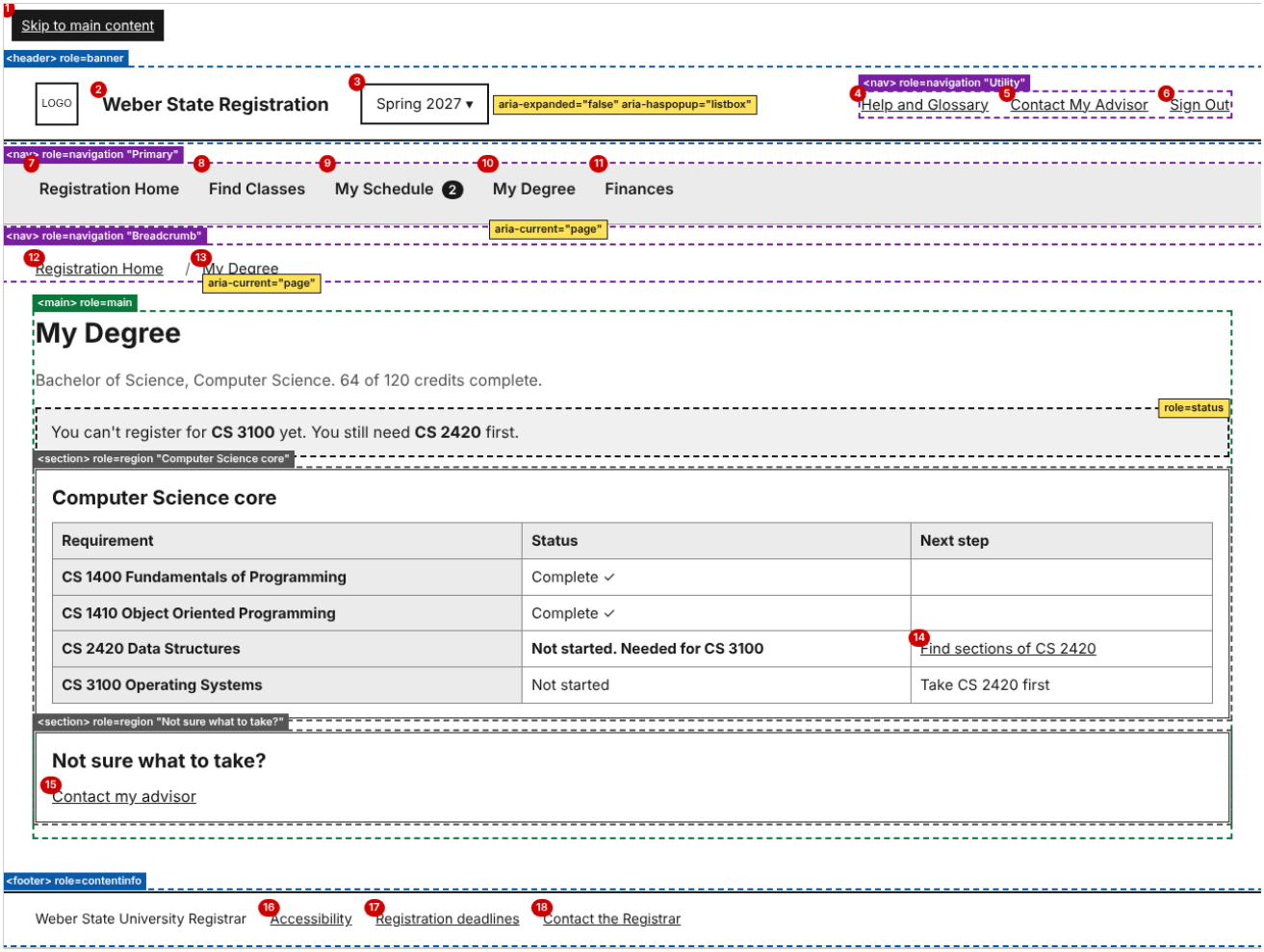


Figure 12. W6 My Degree, annotated.

6. Accessibility First: Semantic Structure and Keyboard Plan

6.1 Semantic HTML Skeleton

Every screen shares this landmark structure. Only `<main>` and the optional `<aside>` change.

```
<body>
  <a class="skip" href="#main">Skip to main content</a>
  <header class="site">                          <!-- banner -->
    <a href="home.html">Weber State Registration</a>
    <button aria-haspopup="listbox" aria-expanded="false">Spring 2027</button>
    <nav aria-label="Utility">...</nav>          <!-- navigation -->
  </header>
  <nav aria-label="Primary">...</nav>            <!-- navigation -->
  <nav aria-label="Breadcrumb"><ol>...</ol></nav><!-- navigation -->
  <main id="main" tabindex="-1">                 <!-- main -->
    <h1>Find Classes</h1>
    <form role="search" aria-label="Class search">...</form>
    <section aria-labelledby="res-h">...</section>   <!-- region -->
  </main>
  <aside aria-labelledby="aside-h">...</aside>   <!-- complementary -->
  <footer class="site">...</footer>              <!-- contentinfo -->
</body>
```

Screen	Landmarks	Labelled regions
W1 Registration Home	banner, 2 navigation, main, contentinfo	Registration date, Holds, My classes, Next steps
W2 and W3 Find Classes	banner, 3 navigation, search, main, complementary, contentinfo	Search results
W4 My Schedule	banner, 3 navigation, main, contentinfo	Pending, Registered, Week view
W5 Registration results	banner, 3 navigation, main, contentinfo	Registered, Not registered
W6 My Degree	banner, 3 navigation, main, contentinfo	Computer Science core, Help

Each `<nav>` has a unique `aria-label`, so a screen reader's landmark list reads "Utility, Primary, Breadcrumb" instead of three identical "navigation" entries. Every page has one `<h1>`, and headings never skip levels.

6.2 Keyboard Tab Order

Tab order follows DOM order, and no positive `tabindex` values are used, so visual order, reading order, and focus order always match (WCAG 2.4.3). The full sequence for all 6 screens is in `05_semantic_html/tab_order.md`. Here is the sequence for the most complex screen:

Table: W2 Find Classes, 31 tab stops

#	Element	Landmark	ARIA
1	Skip to main content (a)	skip link	
2	Weber State Registration (a)	header	
3	Spring 2027 ▾ (button)	header	`aria-haspopup="listbox"` `aria-expanded="false"`
4	Help and Glossary (a)	nav (Utility)	
5	Contact My Advisor (a)	nav (Utility)	
6	Sign Out (a)	nav (Utility)	
7	Registration Home (a)	nav (Primary)	
8	Find Classes (a)	nav (Primary)	`aria-current="page"`
9	My Schedule (2 pending) (a)	nav (Primary)	
10	My Degree (a)	nav (Primary)	
11	Finances (a)	nav (Primary)	
12	Registration Home (a)	nav (Breadcrumb)	
13	Find Classes (a)	nav (Breadcrumb)	`aria-current="page"`
14	Subject (select)	main	
15	Course number (input)	main	`aria-describedby="num-hint"`
16	Morning (input)	main	
17	Afternoon (input)	main	
18	Evening (input)	main	

#	Element	Landmark	ARIA
19	Only open sections (input)	main	
20	Search (button)	main	
21	More filters (days, campus, online) (a)	main	
22	Add to my schedule: CS 2420 Section 001 (button)	main	
23	Details for Section 001 (button)	main	`aria-expanded="false" aria-controls="det-001"`
24	Add to my schedule: CS 2420 Section 002 (button)	main	`aria-describedby="s2-conf"`
25	Details for Section 002 (button)	main	`aria-expanded="false" aria-controls="det-002"`
26	Join waitlist: CS 2420 Section 601 (button)	main	
27	Details for Section 601 (button)	main	`aria-expanded="false" aria-controls="det-601"`
28	Review and register (2) (a)	aside	
29	Accessibility (a)	footer	
30	Registration deadlines (a)	footer	
31	Contact the Registrar (a)	footer	

The sequence keeps every page the same up to stop 13 (skip link, header, utility, primary nav, breadcrumb). A keyboard user learns it once, or skips it with stop 1. Inside `<main>`, the order matches how a student works: filters, then Search, then each result's Add and Details in the order they appear. The My Schedule sidebar comes after the results, so the "Review and register" action is the natural next stop once the student is done adding classes.

6.3 ARIA Roles, States, and Properties

Attribute	Where	Purpose	WCAG
`aria-current="page"`	Active primary nav link, last breadcrumb	Announces where the user is	1.3.1, 2.4.8
`aria-expanded` + `aria-controls`	Semester switcher, every Details button	Announces open or closed and what it controls	4.1.2
`aria-haspopup="listbox"`	Semester switcher	Warns that a list will open	4.1.2
`role="status"` + `aria-live="polite"`	Result count (W2), “Added, not registered yet” (W3), prerequisite notice (W6)	Announces changes without moving focus	4.1.3
`role="alert"`	Registration summary (W5)	Immediately announces “1 of 2 classes registered”	4.1.3, 3.3.1
`aria-describedby`	Course number hint, CRN definition, Register button (drop rule), conflicting Add button (conflict flag), Join waitlist (full message)	Reads the hint or warning with the control	3.3.2
Visually hidden text in buttons	“Add to my schedule: CS 2420 Section 001”, “Details for Section 002”	Every repeated button gets a unique name	2.4.6, 2.5.3
`aria-disabled="true"`	“Added (pending)” on W3	Shows the button is used up without dropping keyboard focus	2.4.3
`tabindex="-1"`	`<main>`, W5 `<h1>`	Target for the skip link and post-submit focus, without adding a tab stop	2.4.1, 2.4.3

6.4 Focus Management

Action	Focus goes to	Screen reader hears
Skip to main content	`<main>`	The page’s `<h1>`

Action	Focus goes to	Screen reader hears
Add to my schedule	Stays on the button, now “Added (pending)”	“CS 2420 Section 001 added to My Schedule. Not registered yet.”
Details	Stays on the button	“expanded” or “collapsed”
Register for 2 classes	W5 `<h1>`	“1 of 2 classes registered. 1 class needs your attention.”

I used `aria-disabled` instead of the `disabled` attribute on the “Added” button on purpose. A truly disabled button loses focus, which would throw a keyboard user back to the top of the page right after adding a class. That is the same kind of failure Phase 1 found (A01).

7. Post-Mortem Reflection

7.1 How did the card sort data challenge my initial assumptions?

Going in, I assumed students think about registration the way the system is built, as a set of tools: a search tool, a schedule, a degree audit, and a billing page. Before running the sort, I sketched a draft sitemap based on the Phase 1 findings. It had “Find Classes” as the search area, a schedule sidebar beside the results mostly as a convenience, and a primary item called “Tuition and Billing.”

The data supported the big pieces but challenged the details. All three participants pulled degree progress and prerequisites into their own group, and all three made a finances group. That confirmed that CatTracks being a separate app is a structural problem, not just a visual one.

The biggest surprise was how experience changed people’s mental models. Both seniors put “see what days and times a class meets” under **schedule**, not under searching, while the freshman put it with search. I had treated time as a search attribute. Experienced students treat it as a planning question: which section fits my week. That changed how I see my own design. The week view sidebar on Find Classes is not an extra. It answers the question students are actually asking, so it stays on that page, and the conflict flags on each section are essential, not optional.

The labels challenged me too. Nobody said “billing.” The open sorters said “finances,” so I renamed the nav item. I had also expected “registration” to be one clear category. P1 split it into doing (enrolling) and checking (account information), while P2 used “register” for almost everything. “Choose a semester” landed in three different groups for three people, which convinced me it shouldn’t live inside any one section.

The data also exposed something I don’t think I fully solved. Students see “register” as one place, but my design spreads adding, submitting, and dropping across pages so each action sits next to its context. I kept that decision on purpose, but it’s the first thing I want to test in Phase 3.

Finally, the difference between a freshman making 7 groups and a senior making 4 showed me there’s no single “correct” number of categories. The right structure has to work for both. Five flat items with no sub-menus was a better compromise than my instinct to add more specific sections.

7.2 How do the ARIA landmarks and tab sequence give screen reader users the same efficiency as sighted users?

A sighted student glances at the page and knows three things right away: where they are, where the main content starts, and what changed after they clicked something. My annotations are meant to give a screen reader user those same three things without reading every word.

Where am I? Sighted users see the underline on the active nav item and the breadcrumb. Screen reader users hear "current page" on that same link, because of `aria-current="page"`. Each `<nav>` has its own label (Utility, Primary, Breadcrumb), so the landmarks list is a real map of the page, not three identical "navigation" entries.

Getting to the content. A sighted user skips the header with their eyes. A keyboard or screen reader user gets the same shortcut from the skip link (stop 1) or by jumping straight to the `main` or `search` landmark. Because the revised IA is flat, with 5 primary items and no sub-menus, there are no nested menus to open and close. Any task is at most two landmark or link jumps from wherever the user is. In the current system, a keyboard user could not even reach the Add buttons.

Knowing what happened. This was the biggest problem in Phase 1. Sighted students missed that Add only made a class pending, and a screen reader user would have had even less to go on. In the redesign:

- Adding a class triggers a polite status message: "Added to My Schedule. Not registered yet."
- Registering moves focus to the results heading and reads the summary as an alert.
- The drop rule is tied to the Register button with `aria-describedby`, so it is read at the exact moment of decision, not buried in legal text before class selection.

Repeated controls. A sighted user can tell which "Add" belongs to which section by position. A screen reader user browsing a list of buttons can't. Visually hidden text gives each button a unique name, like "Add to my schedule: CS 2420 Section 001," so the button list is usable on its own.

Order. Because the tab order is the DOM order, and the DOM order matches the visual layout, a screen reader user and a sighted user move through the page in the same sequence. Nobody gets surprised by focus jumping somewhere unexpected.

8. Documentation and Repository

All raw data and assets are in the `IA_Mapping` directory of https://github.com/Kaden-13/Contextual-Research.

Folder	Contents
`01_card_sort`	Card deck (CSV and printable PDF), facilitator protocol, raw participant data, analysis script, similarity matrix, cluster diagram, consensus summary
`02_sitemap`	Current and revised sitemaps (SVG, PNG, and Graphviz source)
`03_user_flows`	Flow A and Flow B (SVG, PNG, and source)
`04_wireframes`	HTML and CSS wireframe source, annotation script, clean and annotated PNG exports
`05_semantic_html`	Landmark map, ARIA plan, and full tab order for every screen
`06_report`	This report's source and PDF

Every diagram and table can be rebuilt from the raw data with the scripts in each folder.